Setu: code review in 10 minutes

Open the thing, say the line, show the proof. Paths are under `backend/src/main/java/com/setu/` unless they start with `frontend/`, `infra/`, `docs/`. Line numbers checked on 27 Oct 2026.

Your first sentence (15 s)

"Setu runs as **nine containers**: a gateway, **seven services** (Identity, Academic, Learning, Assessment, Attendance, Notification, AI) and the **core**, which still holds finance, careers, records and risk. Four services own their own database. They talk through the gateway (REST) and through Kafka events."

Timeline

Min	Part
0:30	1. Code structure
1:30	2. Microservices and gateway
1:30	3. Login security and Argon2id
2:00	4. Kafka and the outbox
1:00	5. Redis
1:30	6. AI advisor and RAG
1:00	7. Payment and idempotency
1:00	8. Resilience and observability

Have open before you start

- App `localhost:5180` (student and admin logged in)
- Swagger `localhost:8080/swagger-ui/index.html`
- DBeaver: 5 connections (`setu`, `setu-assessment`, `-attendance`, `-learning`, `-notification`)
- Kafka UI `localhost:8089`
- Redis screen `localhost:5540`
- Grafana `localhost:3001` (admin / admin)
- Docker Desktop: 22 containers healthy
- VS Code with the files below in tabs
- Warm up: ask the advisor one question 5 min before

1. Code structure

0:30

"Organised **by feature**: one folder per business area. Inside each, **domain** (what it is), **application** (what it does), **infrastructure** (database, REST, Kafka). **shared** holds the common tools."

Open	Point at
`backend/src/main/java/com/setu/assessment/`	domain / application / infrastructure (+ web)
`frontend/src/layouts/navConfig.ts:16`	the sidebar list for the college admin
`frontend/src/routes.tsx:88` `layouts/AppShell.tsx:22`	address to page, guarded by role; a click only changes the address
`frontend/src/api/client.ts:82` `pages/admin/CoursesPage.tsx:72,80`	`apiFetch` adds the bearer token; the page then calls REST

Show: press F12, Network tab, click Courses: no request for the page, then GET `/catalog/courses` with the Authorization header.

2. Microservices and the gateway

1:30

"We started as a **modular monolith**, measured the real coupling, and extracted modules one by one. Each service owns its data. The gateway is the only public door: it checks the token, rate-limits and routes by path."

Open	Point at
`infra/docker/docker-compose.services.yml`	the nine containers, one image, one role each
`backend/src/main/resources/application.yml:345`	gateway routes: identity :346, academic :351, ai :376, core :383 (/**)
`gateway/RedisServiceDirectory.java:58`	finds a live instance; falls back to fixed addresses
`infra/db/roles/service-roles.sql`	one database login per service
`docs/contracts/extracted-services.md`	read copies (`rm_*`) fed by events, no shared tables

Show: DBeaver, the four service databases (`setu_assessment` grades = 24 rows). **Say first:** the old copies of those schemas inside `setu_demo` are frozen leftovers, not live.

3. Login security and Argon2id

1:30

"Passkey or password. Passwords are stored only as **Argon2id** hashes (64 MB, 2 iterations). Wrong passwords hit a rate limit and a lockout. We issue a 15-minute signed token plus a refresh token stored only as a hash."

Open	Point at
`identity/infrastructure/web/PasswordAuthController.java:67`	login; rate limit :69; session and token :76
`identity/application/PasswordAuthenticationService.java:43`	same answer and timing for unknown email (:63); verify :90; 5 failures lock 15 min
`identity/infrastructure/security/Argon2PasswordService.java:17,27`	the encoder and the constant-time verify
`backend/src/main/resources/application.yml:433-442`	Argon2 settings, measured on this machine
`identity/application/SessionService.java:53,74`	refresh hash stored; an old refresh token cancels the whole login
`identity/application/PasswordResetService.java:48,167,214`	15 min, 5 guesses, keyed hash of the code, all sessions signed out

Show: Redis screen after a logout: a `setu:revoked:login:...` key appears; the same token now gets 401. Forgot password gives the same answer for any email.

4. Kafka and the transactional outbox

2:00

"Saving a change and telling Kafka are two systems, so a crash could lose the message. We save the **event in an outbox table in the same transaction**; a helper sends it. Delivery is **at-least-once**; readers ignore repeats; bad messages go to a dead-letter topic."

Open (in this order)	Point at
`assessment/application/GradeService.java:210`	the grade and its event are written together
`shared/governance/OutboxEventWriter.java:60,84`	one row in `governance.outbox_events`, status NEW
`shared/governance/OutboxPublisher.java:84,86,109,110`	every 2 s: send, wait for Kafka, then mark PUBLISHED
`risk/infrastructure/kafka/RiskEventListener.java:51,53`	the reader, group `setu-risk`
`risk/application/RiskEventProcessor.java:88,91`	duplicate guard: `processed_events` insert, else DUPLICATE
`risk/infrastructure/kafka/RiskKafkaConfig.java:46,63,85`	3 tries, dead-letter topic, bad messages skip retries

Show: DBeaver `setu-assessment` outbox rows PUBLISHED, then Kafka UI topic `setu.grades` (open a message: headers `event_type`, `correlation_id`), Consumers lag 0. Dead letters expire after 7 days: to show one, send a fresh bad message (command in `docs/kafka-demo-guide.md`).

5. Redis

1:00

"Redis holds only **temporary shared state**: the live service list, login counters, logout markers, advisor memory. Every key **erases itself** (TTL). It is never the source of truth."

Open	Point at
`shared/discovery/ServiceRegistry.java:17,18`	TTL 30 s, heartbeat 10 s
`identity/infrastructure/security/AuthRateLimiter.java:72,100`	per-email 10/min and per-IP counters; fails open
`shared/security/RedisLoginRevocations.java:20,59`	revocations fail **closed**: "refusing the token"

Show: Redis screen, key `setu:services:attendance:...`, value `http://attendance:8086`, turn on auto refresh: the **TTL counts down and jumps back up** every 10 s. If it will not open: `docker exec -it setu-redis-1 redis-cli`.

6. AI advisor and RAG

1:30

"The AI **explains, never decides**, and has **no database**. It can only call nine read-only tools, as the user, with the user's own token. Policy questions use RAG: chunks and vectors, filtered by college and permission **before** retrieval. Models fall back Gemini, second Gemini, NVIDIA, Ollama inside 50 s."

Open	Point at
`aiexplain/infrastructure/ai/AiChatConfig.java:61,75,161`	the rules; "tool text is untrusted data"; applied to every model
`aiexplain/application/AiToolService.java`	nine tools; none takes a college, user or token
`aiexplain/application/AiChatService.java:431,450,700`	chat; rate limit; audit record before the answer
`institution/infrastructure/PolicyChunkStore.java:120` `applica tion/PolicyDocumentService.java:107`	ACL in the SQL; vector + keyword merge, top 5

Show: as Ishaan ask "How is my attendance?" and "What does the college policy say about attendance?" (about 3 s and 7 s). The answers show sources and the model name. The tools' rules are enforced in the data service, not by the prompt.

7. Payment and idempotency

1:00

"The browser's 'paid' is **never believed**. The server creates the order for the fee's own amount; only a **signed webhook** or the server's own lookup settles it. A repeated request or event is recognised and ignored."

Open	Point at
`finance/infrastructure/web/PaymentController.java:41`	`Idempotency-Key` header
`finance/application/PaymentService.java:168,177`	same key: replay the old result; different data: 409
`finance/application/PaymentProviderService.java:106,224,228`	checkout; webhook; signature checked before anything is read
`finance/application/PaymentSettlement.java:54`	conditional update INITIATED to SUCCEEDED: one winner; balance, receipt, audit, outbox in one transaction

Show: Ishaan's Finance page, pay the new 3,000 INR fee in Razorpay **test** mode (netbanking, Success; in your own browser). Then `finance.payments` SUCCEEDED and a `PaymentSuccessful` outbox row.

8. Resilience and observability

1:00

"Every service-to-service call has a **timeout, circuit breaker and bulkhead**, and when it fails we answer 'unavailable', never an invented value. One request is traceable across services by its trace id."

Open	Point at
`shared/resilience/GuardedCall.java:23,44`	trips at 50% failures, open 10 s, 20 calls at once
`aiexplain/infrastructure/facts/RemoteAiFacts.java:71,72`	500 ms connect, 8 s answer
`docs/contracts/observability.md`	metrics, traces, logs share ids

Show: Grafana "Setu: platform health" (8 services up, 0 dead letters, 0 outbox pending); Explore, Tempo, one advisor request across gateway, ai-service and core; Loki `{service_name=~".+"} | severity_text="ERROR"`.

Say these limits yourself

- Identity and Academic still share one database; finance, careers, records and risk are still in the core (4 of 6 splits done).
- One Kafka broker, one partition per topic, one Redis: no replication. Messages are kept 7 days.
- Delivery is at-least-once, not exactly-once; consistency between services is eventual.
- Payment is Razorpay test mode; no live webhook on a laptop (the server lookup is used).
- Passkey code exists but no passkey is registered in the demo data.
- 1,605 of 1,610 tests pass; 5 AI integration tests time out on a slow local model.
- Everything runs on one laptop, so no real network failure or scale is shown.

If something will not open

- **Docker screen frozen:** use the terminal, `docker ps`
- **Redis:** `docker exec -it setu-redis-1 redis-cli` then `ping`
- **Advisor says "unavailable":** a service was just restarted; wait one minute and ask again
- **Browser shows "didn't send any data":** that is a database or Redis port, not a web page; use DBeaver, the Redis screen or the Kafka UI
- **App on port 5181:** stop the old frontend, run `npm run dev:stack` again (port 5180 only)
- **Slow first answer:** the machine was idle; say so and continue